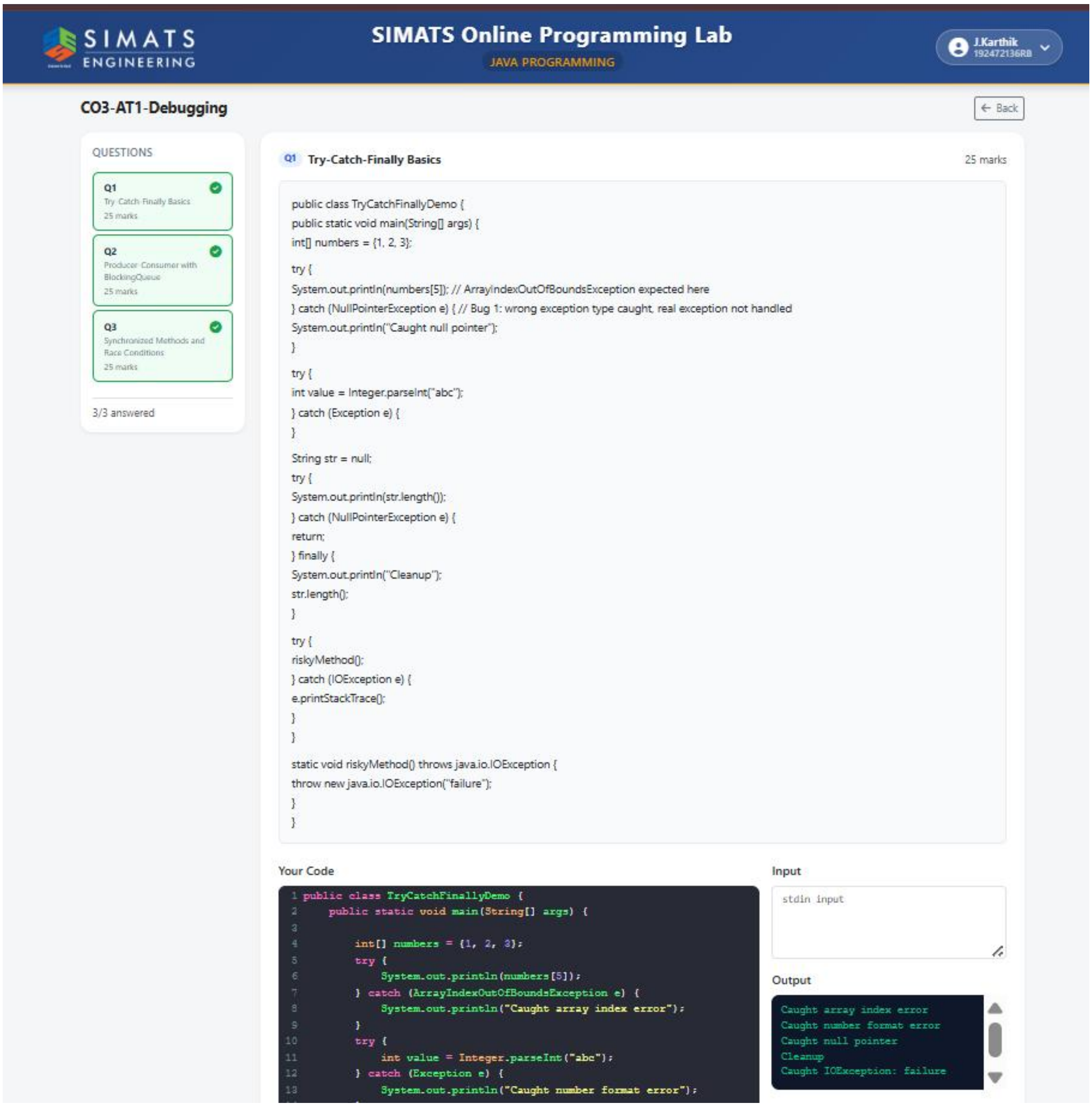
J KARTHIK

192472136

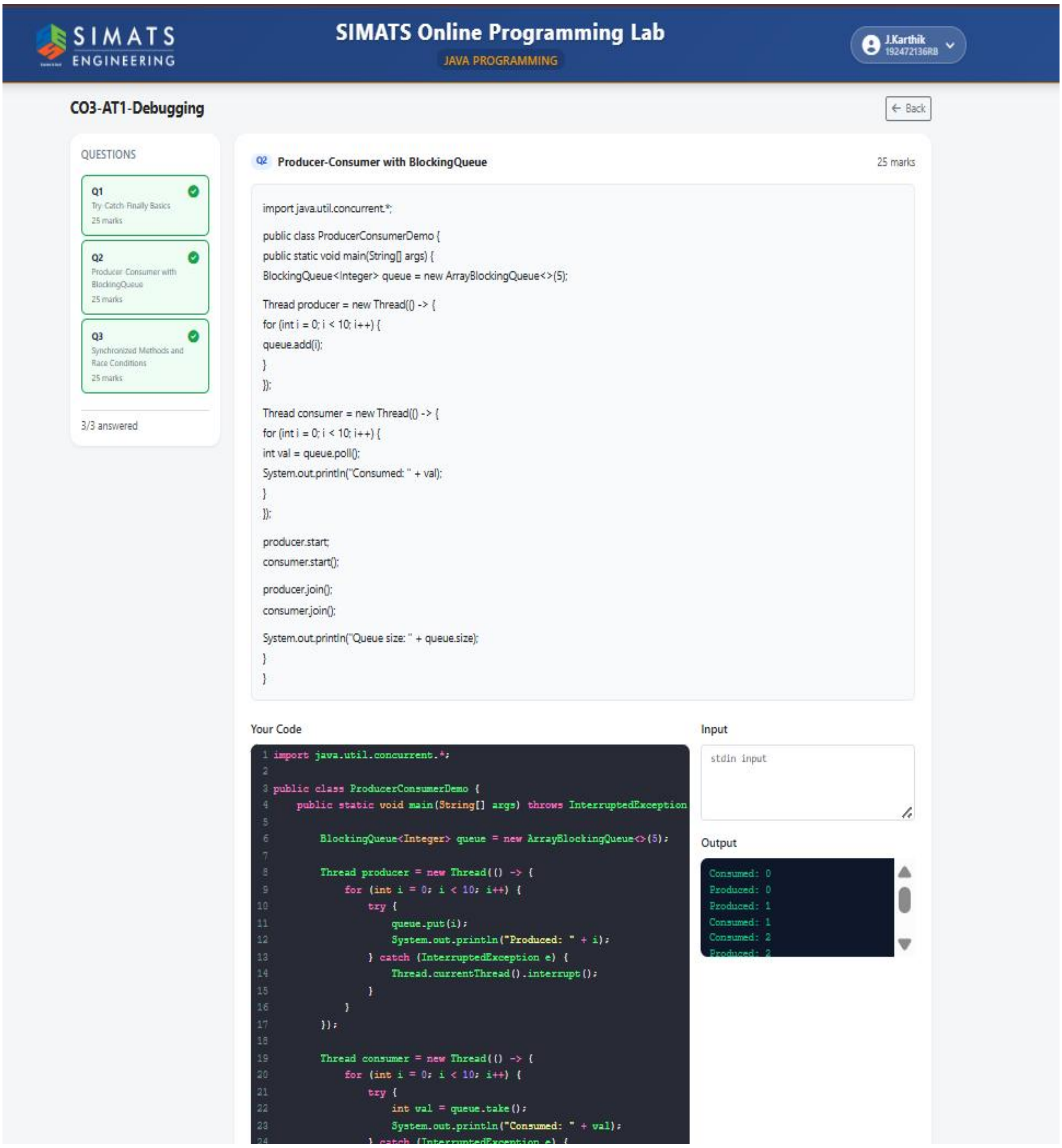
CSA0911

JAVA

1. Try – Catch – Finally Basks



2. Producer – Consumer with BlockingQueue



SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

J.Karthik 192472136RB

CO3-AT1-Debugging

← Back

QUESTIONS

Q1
Try Catch Finally Basics
25 marks

Q2
Producer Consumer with BlockingQueue
25 marks

Q3
Synchronized Methods and Race Conditions
25 marks

3/3 answered

Q2 Producer-Consumer with BlockingQueue

25 marks

```
import java.util.concurrent.*;
public class ProducerConsumerDemo {
public static void main(String[] args) {
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);
Thread producer = new Thread(() -> {
for (int i = 0; i < 10; i++) {
queue.add(i);
}
});
Thread consumer = new Thread(() -> {
for (int i = 0; i < 10; i++) {
int val = queue.poll();
System.out.println("Consumed: " + val);
}
});
producer.start;
consumer.start();
producer.join();
consumer.join();
System.out.println("Queue size: " + queue.size);
}
}
```

Your Code

```
import java.util.concurrent.*;

public class ProducerConsumerDemo {
    public static void main(String[] args) throws InterruptedException

        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);

        Thread producer = new Thread(() -> {
            for (int i = 0; i < 10; i++) {
                try {
                    queue.put(i);
                    System.out.println("Produced: " + i);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });

        Thread consumer = new Thread(() -> {
            for (int i = 0; i < 10; i++) {
                try {
                    int val = queue.take();
                    System.out.println("Consumed: " + val);
                } catch (InterruptedException e) {
```

Input

stdin input

Output

```
Consumed: 0
Produced: 0
Produced: 1
Consumed: 1
Consumed: 2
Produced: 2
```

3. Synchronized Methods and Race Conditions